Supplementary Information: Predicting Scintillator Light Yield Using Physics-Informed Machine Learning

Chaitanya Sagar Tadi, Rabiatu Mahadi, Augustine Twumasi,
Halima Bensmail, Salawu Omotayo Akande, and Othmane Bouhali

S1 Supplementary Information

S1.1 Dataset Curation and Limitations

This work is based on a meta-analysis of 295 unique inorganic scintillator materials compiled from existing literature. While the machine learning models demonstrate strong generalization on held-out test data, it is important to contextualize their development by discussing the inherent limitations of any study based on a literature-compiled dataset. Several key challenges represent potential sources of uncertainty:

- **Label noise** in the `lightyield` target variable, which stems from differences in experimental equipment, photodetector quantum efficiency, shaping times, and calibration standards across publications;
- **Measurement heterogeneity** in the input features, where properties such as host band gap (E_g) may be reported from either experimental optical absorption/reflection measurements or density functional theory (DFT) calculations; and
- **Publication bias**, which naturally skews literature data toward successful, higher-yielding materials, potentially under-representing poorly performing or non-luminescent host-activator combinations.

Together, these factors highlight an inherent extrapolation risk when screening completely uncharted chemical classes. Consequently, the model's quantitative reliability is highest when interpolating within the physicochemical descriptor space spanned by the training set, whereas predictions for novel compounds with extreme descriptor values should be interpreted as relative prioritization rankings. Furthermore, because the empirical distributions of the target property and several key descriptors are strongly non-Gaussian (Figure S1), non-linear ensemble tree methods and logarithmic target scaling are essential for robust modeling.

S2 Data Preprocessing, Imputation, and Transformation

The curated dataset (`all_scintillators_cleaned2.csv`) serves as the foundation for the modeling workflow. A standardized multi-step preprocessing pipeline was applied to ensure data quality, physical consistency, and reproducibility.

S2.1 Data Cleaning

1. **Duplicate Removal:** Any fully duplicated rows were removed.

2. **Target Variable Check:** Entries lacking a measured `lightyield` value were excluded from the supervised learning workflow, yielding 295 complete compound entries across diverse host matrices (Figure S2).

3. **Feature Pruning:** While exploratory distributions were evaluated across the initial 18 raw physicochemical descriptors and the target variable (Figure S1), the electronic and ionic dielectric constant columns (`av_el_diel_cons` and `av_ion_diel_cons`) were excluded prior to final model training due to high missingness across literature sources, leaving 16 base descriptors.

S2.2 Median Imputation

Missing entries in the remaining 16 numeric base features (excluding the `lightyield` target) were handled using median imputation. For any feature containing missing values, empty cells were populated with that feature's median value across the dataset. Median imputation was selected over mean imputation to prevent skewed feature distributions and extreme outliers from distorting the imputed values.

S2.3 Data Distribution and Log Transformation

The experimental `lightyield` target variable is heavily right-skewed, spanning from $< 1,000$ ph/MeV to $> 100,000$ ph/MeV (Figure S3, left). To stabilize variance and transform the target distribution into a more Gaussian-like form (Figure S3, right), a natural logarithmic transformation $y_{\text{transformed}} = \ln(1 + y_{\text{actual}})$ (`np.log1p`) was applied prior to training. All models were trained and tuned on this log-transformed scale. For held-out test evaluation, predictions were converted back to the original photon-yield scale via $y_{\text{pred}} = \exp(\hat{y}_{\text{transformed}}) - 1$ (`np.expm1`) to compute R^2, RMSE, and MAE in physical units (ph/MeV).

S2.4 Physics-Informed Feature Engineering

Following cleaning and imputation, three domain-specific interaction features were engineered from the 16 base descriptors to yield the final set of 19 features:

1. `electronegativity_mismatch`: $|\texttt{av.ele(+)} - \texttt{av.ele(-)}|$
2. `size_mismatch`: $\dfrac{|\texttt{emp_rad(+)} - \texttt{emp_rad(-)}|}{\texttt{emp_rad(+)} + \texttt{emp_rad(-)} + \epsilon}$
3. `bandgap_doping_ratio`: $\dfrac{\texttt{host_band_gap}}{\texttt{dop_conc} + \epsilon}$ (where $\epsilon = 10^{-6}$ prevents division by zero for intrinsic/undoped scintillators).

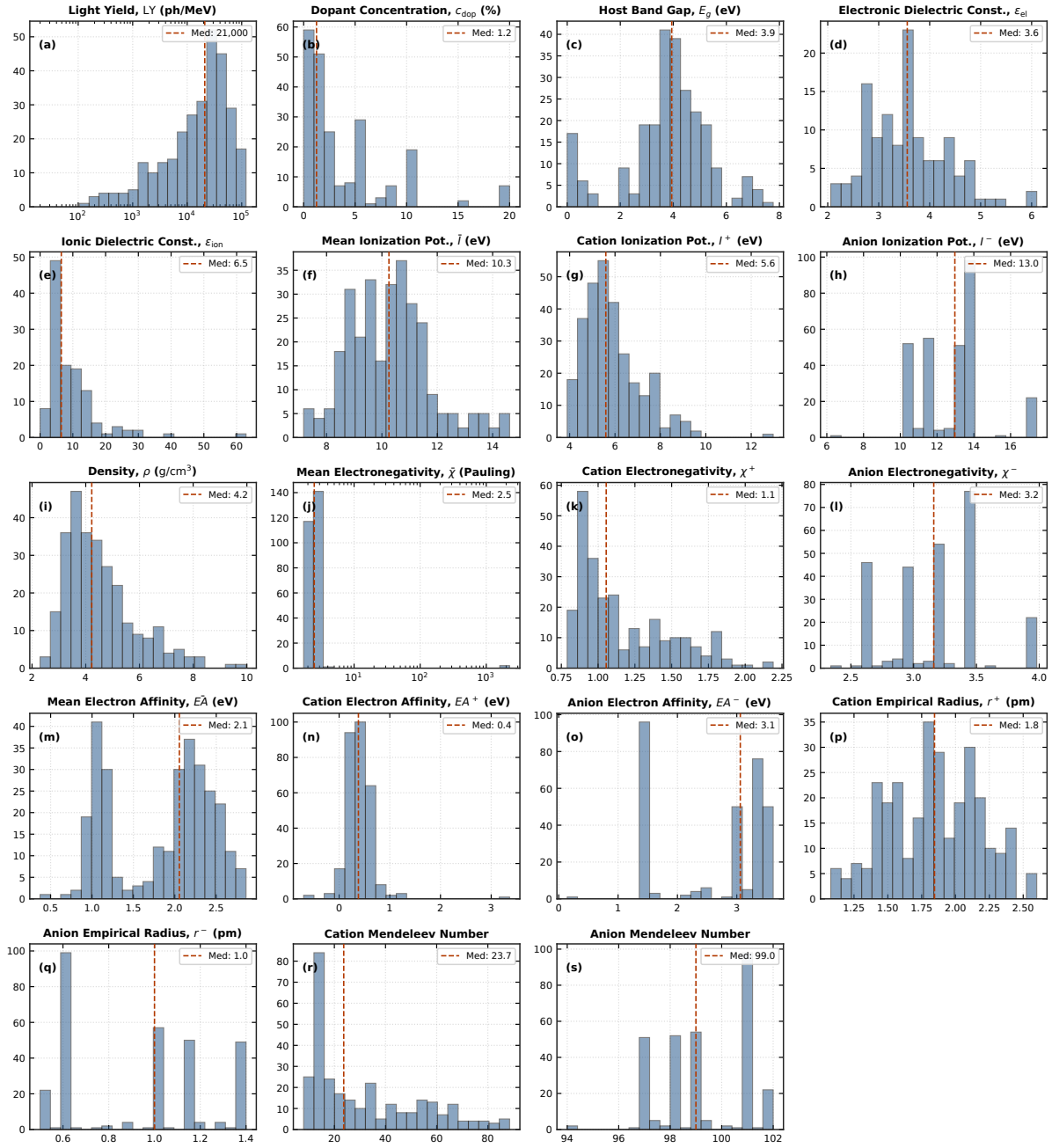


Figure S1: Empirical distributions across all numerical descriptors and the target light yield. Heavily skewed quantities spanning orders of magnitude are shown with dashed red vertical lines marking the sample median for each property.

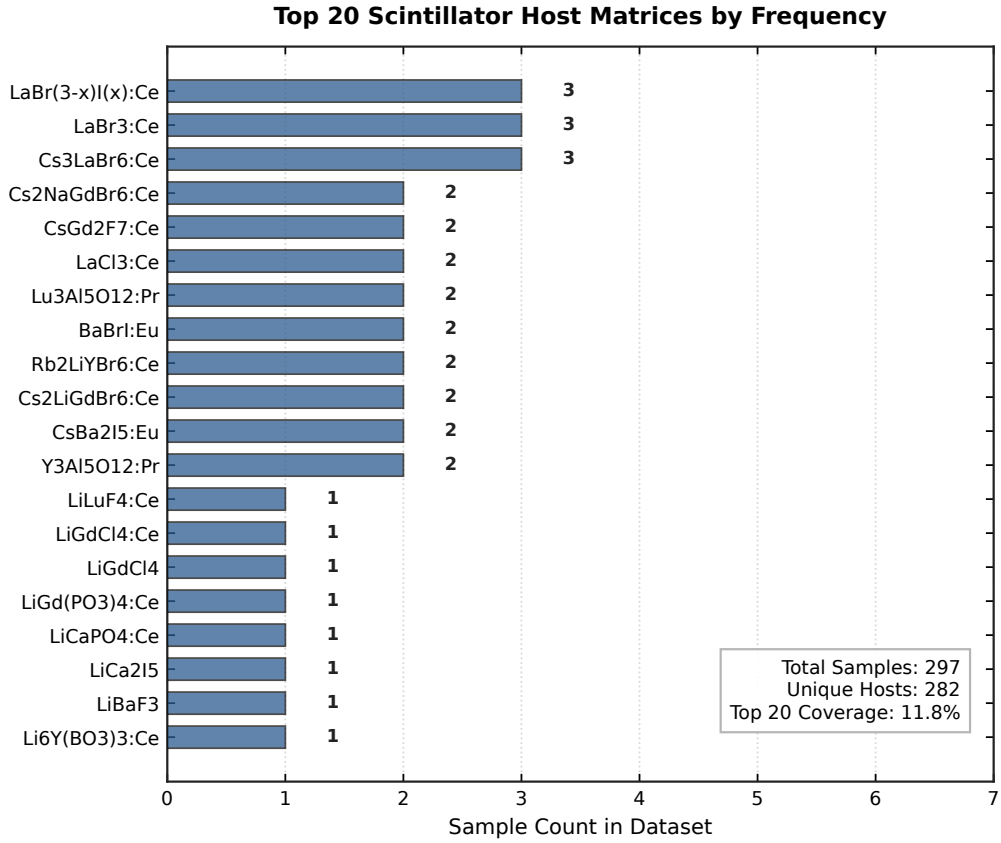


Figure S2: Ranked sample frequency across the top 20 scintillator host matrices in the curated database. Explicit sample counts are labeled alongside each bar, with the inset detailing overall host diversity.

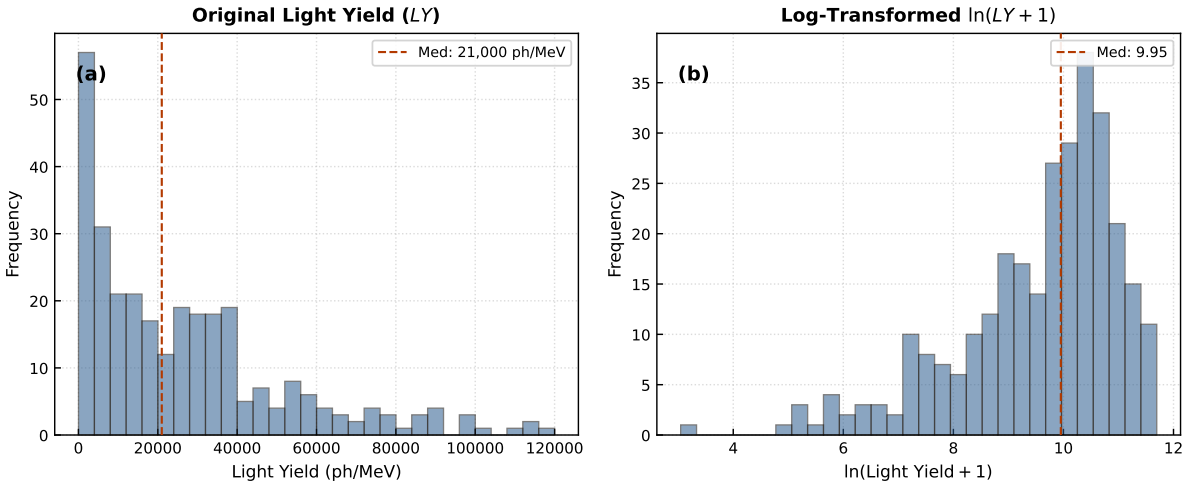


Figure S3: Effect of natural logarithmic transformation ($\ln(1 + \mathrm{LY})$) on the right-skewed experimental light yield distribution.

S3 Physicochemical Feature Definitions

The 16 base physicochemical features used alongside the 3 engineered PIML features are defined in Table S1.

Table S1: Definitions of the 16 base physicochemical descriptors.

Feature	Definition
`dop_conc`	Dopant/activator concentration (mol%) as reported in the literature.
`host_band_gap`	Electronic band gap of the host material (eV).
`av_ion_pot`	Composition-averaged first ionization potential of all constituent atoms (eV).
`ion_pot(+)`	Average first ionization potential of the cationic elements (eV).
`ion_pot(-)`	Average first ionization potential of the anionic elements (eV).
`Density (g/cm`^{3}`)`	Mass density of the host scintillator material (g/cm^3).
`av_electroneg`	Composition-averaged Pauling electronegativity ($\bar{\chi}$) of all constituent atoms.
`av.ele(+)`	Average Pauling electronegativity of the cationic elements (χ^+).
`av.ele(-)`	Average Pauling electronegativity of the anionic elements (χ^-).
`av_elec_aff`	Composition-averaged electron affinity of all constituent atoms (eV).
`elec_aff(+)`	Average electron affinity of the cationic elements (eV).
`elec_aff(-)`	Average electron affinity of the anionic elements (eV).
`emp_rad(+)`	Average empirical atomic/ionic radius of the cationic elements (Å).
`emp_rad(-)`	Average empirical atomic/ionic radius of the anionic elements (Å).
`mendeleev(+)`	Average Mendeleev number of the cationic elements.
`mendeleev(-)`	Average Mendeleev number of the anionic elements.

S4 Model Hyperparameters and SHAP Interpretability

All models were implemented in Python using `scikit-learn` and `xgboost`. The dataset was partitioned into an 80% training set and a 20% held-out test set (`random_state=42`), and hyperparameter optimization was performed via 5-fold cross-validated randomized search (`RandomizedSearchCV`) on the training set. The optimized `GradientBoostingRegressor` achieved the highest test-set generalization ($R^2 = 0.962$, RMSE $= 5,037$ ph/MeV, MAE $= 3,116$ ph/MeV) and was selected as the final model. Its optimal hyperparameters and search bounds are summarized in Table S2.

Table S2: Hyperparameter search space and optimal configuration for the selected `GradientBoostingRegressor` model.

Model	Hyperparameter	Search Space	Selected Value
`GradientBoostingRegressor`	`n_estimators`	$[100, 200, 300]$	200
	`learning_rate`	$[0.01, 0.05, 0.1]$	0.1
	`max_depth`	$[3, 5, 7]$	5
	`subsample`	$[0.8, 0.9, 1.0]$	0.8
	`loss`	Fixed	`squared_error`
	`random_state`	Fixed	42

S4.1 SHAP Summary Analysis for the Best Model (Gradient Boosting)

A limitation of traditional impurity-based feature importance scores in tree ensembles is that they do not reveal the *directionality* of a feature's effect—that is, whether a high or low value

of a given physicochemical property increases or decreases predicted light yield. To resolve both global importance and instance-level directional behavior, we employed the SHAP (SHapley Additive exPlanations) framework [3], which improves upon local surrogate methods such as LIME [4] by guaranteeing local accuracy, missingness, and consistency.

Grounded in cooperative game theory [2, 5, 1], the exact Shapley value ϕ_i representing the marginal contribution of feature i across the set of all d features Q is computed as:

$$\phi_i = \sum_{S \subseteq Q \setminus \{i\}} \frac{|S|!\,(|Q| - |S| - 1)!}{|Q|!} \left[H_{S \cup \{i\}}(x_{S \cup \{i\}}) - H_S(x_S) \right] \tag{1}$$

where S denotes any subset of features excluding feature i, and $H_S(x_S)$ is the model prediction evaluated over subset S. Rather than evaluating all 2^d possible feature coalitions directly, TreeSHAP [3] computes exact Shapley values in polynomial time by tracking decision paths through the ensemble trees.

Figure S4 presents (a) the global feature importance ranking (mean(|SHAP|)) and (b) the directional SHAP beeswarm summary plot on the target scale $\ln(1 + \mathrm{LY})$ for the optimized Gradient Boosting model. Each point in Figure S4b represents an individual scintillator compound, where color encodes the relative magnitude of the feature value (red = high, blue = low) and horizontal position indicates the signed SHAP contribution toward higher (positive) or lower (negative) predicted light yield:

- **Electronic Polarizability and Band-Edge Promoters:** Moderate-to-low values of average electronegativity ($\bar{\chi}$, blue points at positive SHAP values) and lower anionic Mendeleev numbers (`mendeleev-`), combined with higher mean electron affinity ($\overline{\mathrm{EA}}$, red points on the right) and larger anion empirical radii (r^-, red points on the right), strongly shift predictions toward higher light yields. Physically, this reflects the superior conversion efficiency and smaller band gaps of heavy-halide (e.g., I^-, Br^-, Cl^-) and chalcogenide lattices compared to strongly electronegative, wide-bandgap fluorides and oxides (red $\bar{\chi}$ points clustered at negative SHAP values).

- **Host–Activator Balance ($E_g/\mathbf{Conc_{dop}}$):** Elevated values of the engineered bandgap-to-doping ratio yield positive SHAP contributions (red points extending to the right), capturing the requirement of sufficient host excitation energy paired with moderated activator concentrations that avoid self-absorption and concentration quenching.

- **Steric Strain Suppressors (Δr):** High values of ionic radius mismatch (Δr, red points clustered on the left at negative SHAP values) consistently depress predicted light yield, capturing the detrimental role of lattice distortion, dopant segregation, and strain-induced non-radiative trapping centers.

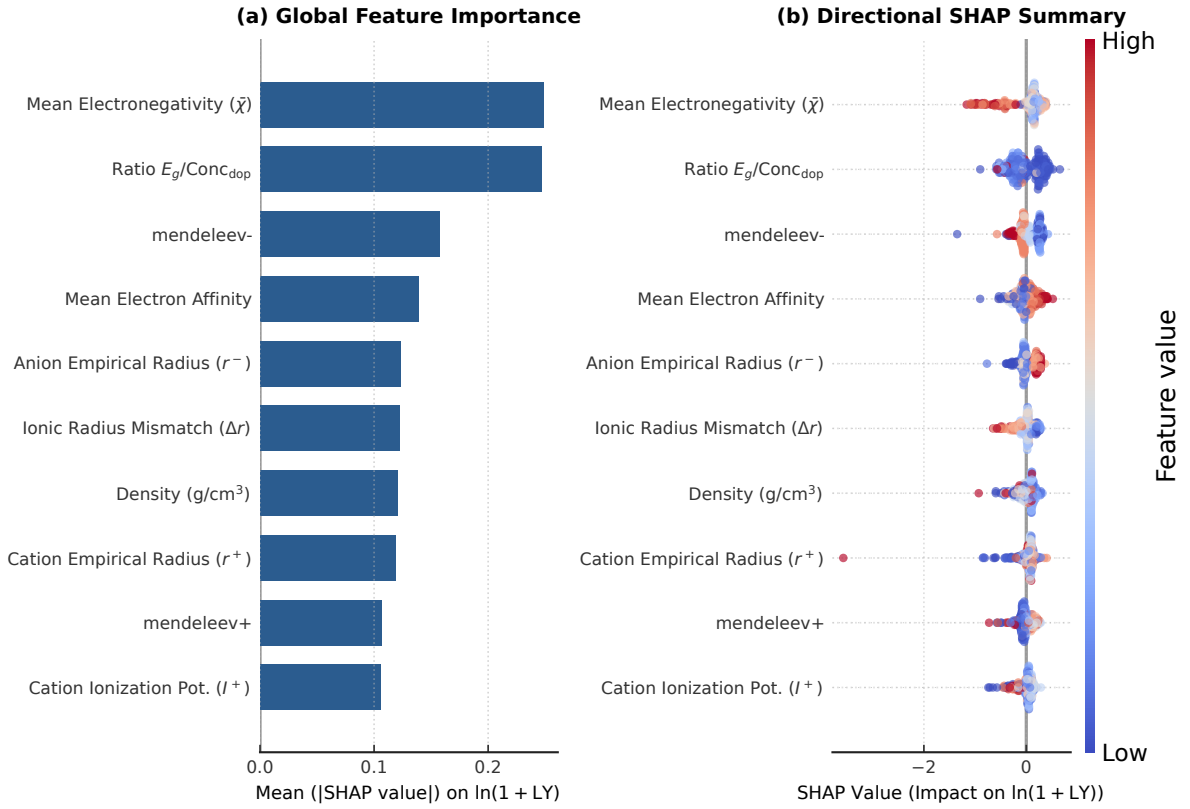


Figure S4: (a) Global feature importance ranking for the top 10 descriptors from the optimized Gradient Boosting model, evaluated by the mean absolute SHAP value (mean(|SHAP|)) on $\ln(1 + \mathrm{LY})$. (b) Directional SHAP beeswarm summary plot illustrating how individual feature values influence predicted light yield on the $\ln(1 + \mathrm{LY})$ scale. Each point represents a single compound, with color indicating high (red) or low (blue) descriptor values.

S4.2 Reproducible Python Workflow

The complete training, cross-validated hyperparameter optimization, held-out test evaluation, and physics benchmark script is provided below and at https://github.com/chaitanyasagartadi/PIML-Scintillators.

```
#
    ================================================================================

# UNIFIED ANALYSIS PIML scintillator Light Yield
#
    ================================================================================

import os
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
from sklearn.model_selection import RandomizedSearchCV
from sklearn.feature_selection import RFECV
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import xgboost as xgb
import shap

warnings.filterwarnings('ignore')

# --- Global Publication Aesthetics & Theme ---
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.sans-serif': ['DejaVu Sans', 'Arial', 'Helvetica'],
    'pdf.fonttype': 42,
    'ps.fonttype': 42,
    'font.size': 9.5,
    'axes.labelsize': 10,
    'axes.titlesize': 11,
    'xtick.labelsize': 9,
    'ytick.labelsize': 9,
    'axes.linewidth': 0.9,
    'axes.edgecolor': '#222222',
    'xtick.direction': 'in',
    'ytick.direction': 'in',
    'xtick.top': True,
    'ytick.right': True,
    'figure.dpi': 300,
    'savefig.dpi': 300,
    'savefig.bbox': 'tight'
})

#
    ================================================================================

# 0. CONFIGURATION & DIRECTORY SETUP
#
    ================================================================================

DATA_PATH = '/content/drive/MyDrive/PET_work/all_scintillators.csv'
OUTPUT_DIR = '/content/drive/MyDrive/PET_work/figures_pub/'
os.makedirs(OUTPUT_DIR, exist_ok=True)

print("--- 1. LOADING AND STANDARDIZING DATASET ---")
```

```
try:
    df = pd.read_csv(DATA_PATH)
except FileNotFoundError:
    print(f"Error: Could not find raw file at {DATA_PATH}. Please verify your Drive path.")
    exit()

# Standardize column names to snake_case
df.columns = (
    df.columns.str.strip()
    .str.lower()
    .str.replace(' ', '_', regex=False)
    .str.replace('(', '', regex=False)
    .str.replace(')', '', regex=False)
    .str.replace('/', '', regex=False)
)

# ============================================================================

# 2. RIGOROUS DATA CLEANING & IMPUTATION (N = 295)
# ============================================================================

print("\n--- 2. DATA CLEANING & PREPROCESSING ---")
df_cleaned = df.copy()
initial_count = len(df_cleaned)
df_cleaned.drop_duplicates(inplace=True)
df_cleaned.dropna(subset=['lightyield'], inplace=True)

# Drop high-missingness columns (~58.6% dielectric constants)
cols_to_drop = ['av_el_diel_cons', 'av_ion_diel_cons']
df_cleaned.drop(columns=[c for c in cols_to_drop if c in df_cleaned.columns], inplace=True)

# Median imputation for remaining features
numeric_cols_imp = df_cleaned.select_dtypes(include=np.number).columns.tolist()
if 'lightyield' in numeric_cols_imp:
    numeric_cols_imp.remove('lightyield')
for col in numeric_cols_imp:
    if df_cleaned[col].isnull().any():
        df_cleaned[col].fillna(df_cleaned[col].median(), inplace=True)

# ============================================================================

# 3. PHYSICS-INFORMED FEATURE ENGINEERING
# ============================================================================

print("\n--- 3. PHYSICS-INFORMED FEATURE ENGINEERING ---")
epsilon = 1e-6
df_cleaned['electronegativity_mismatch'] = np.abs(df_cleaned['av.ele+'] - df_cleaned['av.ele-'])
df_cleaned['size_mismatch'] = np.abs(df_cleaned['emp_rad+'] - df_cleaned['emp_rad-']) / (
    df_cleaned['emp_rad+'] + df_cleaned['emp_rad-'] + epsilon
)
df_cleaned['bandgap_doping_ratio'] = df_cleaned['host_band_gap'] / (df_cleaned['dop_conc'] + epsilon)

# ============================================================================
```

```
# 4. MODEL TRAINING, RFECV, & HYPERPARAMETER TUNING
#    ============================================================================

print("\n--- 4. MODEL TRAINING & SELECTION PIPELINE ---")
features_to_exclude = ['lightyield', 'scintillator', 'mechanism', 'reference']
X = df_cleaned.drop(columns=[col for col in features_to_exclude if col in
    df_cleaned.columns])
X = X.select_dtypes(include=np.number)
y = np.log1p(df_cleaned['lightyield'])

models_dict = {
    'RandomForest': {
        'model': RandomForestRegressor(random_state=42),
        'params': {
            'n_estimators': [100, 200, 300],
            'max_depth': [10, 20, 30, None],
            'min_samples_split': [2, 5, 10],
            'min_samples_leaf': [1, 2, 4],
            'max_features': ['sqrt', 'log2', 1.0]
        }
    },
    'GradientBoosting': {
        'model': GradientBoostingRegressor(random_state=42),
        'params': {
            'n_estimators': [100, 200, 300],
            'learning_rate': [0.01, 0.05, 0.1],
            'max_depth': [3, 5, 7],
            'subsample': [0.8, 0.9, 1.0]
        }
    },
    'XGBoost': {
        'model': xgb.XGBRegressor(random_state=42, objective='reg:squarederror'
    ),
        'params': {
            'n_estimators': [100, 200, 300],
            'learning_rate': [0.01, 0.05, 0.1],
            'max_depth': [3, 5, 7],
            'subsample': [0.8, 0.9, 1.0],
            'colsample_bytree': [0.8, 0.9, 1.0]
        }
    }
}

results = {}
for name, cfg in models_dict.items():
    print(f"\nProcessing model: {name}")
    selector = RFECV(estimator=cfg['model'], step=1, cv=5, scoring='r2', n_jobs
    =-1)
    selector.fit(X, y)
    X_sel = X.loc[:, selector.support_]

    rand_search = RandomizedSearchCV(
        estimator=cfg['model'], param_distributions=cfg['params'],
        n_iter=25, cv=5, random_state=42, n_jobs=-1, scoring='r2'
    )
    rand_search.fit(X_sel, y)
    best_est = rand_search.best_estimator_

    y_pred_log = best_est.predict(X_sel)
    y_pred_orig = np.expm1(y_pred_log)
    y_actual_orig = df_cleaned['lightyield'].values
```

```
156
157     r2_val = r2_score(y_actual_orig, y_pred_orig)
158     rmse_val = np.sqrt(mean_squared_error(y_actual_orig, y_pred_orig))
159
160     results[name] = {
161         'best_model': best_est,
162         'r2_score': r2_val,
163         'rmse': rmse_val,
164         'y_pred': y_pred_orig,
165         'y_actual': y_actual_orig,
166         'band_gap': df_cleaned['host_band_gap'].values,
167         'X_selected': X_sel
168     }
169     print(f"--> {name} Optimized | R2: {r2_val:.3f} | RMSE: {rmse_val:.1f} ph/
    MeV")
170
171 # Identify best performing model dynamically
172 best_model_name = max(results, key=lambda k: results[k]['r2_score'])
173 best_data = results[best_model_name]
174
175 #
    ================================================================================

176 # 5. AUTOMATED GENERATION OF ALL UNIQUE PUBLICATION FIGURES
177 #
    ================================================================================

178 print("\n--- 5. GENERATING PUBLICATION-GRADE FIGURES ---")
179
180 # --- Figure S1: Feature Distributions ---
181 canonical_labels = {
182     'lightyield': r'Light Yield, $\mathrm{LY}$ (ph/MeV)',
183     'dop_conc': r'Dopant Concentration, $c_{\mathrm{dop}}$ (%)',
184     'host_band_gap': r'Host Band Gap, $E_g$ (eV)',
185     'density_gcc': r'Density, $\rho$ ($\mathrm{g/cm^3}$)',
186     'electronegativity_mismatch': r'Electronegativity Mismatch, $\Delta\chi$',
187     'size_mismatch': r'Ionic Radius Mismatch, $\Delta r$',
188     'bandgap_doping_ratio': r'Ratio $E_g / c_{\mathrm{dop}}$'
189 }
190
191 num_cols = df_cleaned.select_dtypes(include=np.number).columns.tolist()
192 n_cols_grid = 4
193 n_rows_grid = int(np.ceil(len(num_cols) / n_cols_grid))
194 fig, axes = plt.subplots(n_rows_grid, n_cols_grid, figsize=(11.5, 2.5 *
    n_rows_grid))
195 axes = np.array(axes).flatten()
196
197 for idx, col in enumerate(num_cols):
198     ax = axes[idx]
199     srs = df_cleaned[col].dropna()
200     lbl = canonical_labels.get(col, col.replace('_', ' ').title())
201     bins = np.logspace(np.log10(srs.min()), np.log10(srs.max()), 22) if (srs.
    min() > 0 and srs.max()/srs.min() > 50) else 20
202     if srs.min() > 0 and srs.max()/srs.min() > 50:
203         ax.set_xscale('log')
204
205     ax.hist(srs, bins=bins, color='#2b5c8f', alpha=0.55, edgecolor='#111111',
    linewidth=0.75, zorder=2)
206     med = srs.median()
207     med_str = f"Med: {med:.1f}" if med < 1000 else f"Med: {med:,.0f}"
208     ax.axvline(med, color='#b33a00', linestyle='--', linewidth=1.1, label=
    med_str, zorder=3)
```

```
    ax.text(0.04, 0.90, f"({chr(97 + idx)})", transform=ax.transAxes, fontsize=8.5, fontweight='bold', va='top')
    ax.set_title(lbl, fontsize=8.5, fontweight='semibold', pad=5)
    ax.tick_params(labelsize=7.5, direction='in')
    ax.grid(True, linestyle=':', alpha=0.45, zorder=1)
    ax.legend(fontsize=7.0, loc='upper right', framealpha=0.9)

for j in range(len(num_cols), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
for ext in ['pdf', 'png']:
    plt.savefig(os.path.join(OUTPUT_DIR, f'Figure_S1_feature_distributions.{ext}'), format=ext)
plt.close()

# --- Figure S2: Top Scintillator Host Compound Frequency ---
host_col = 'scintillator' if 'scintillator' in df_cleaned.columns else 'Scintillator'
top_hosts = df_cleaned[host_col].value_counts().head(20)
fig, ax = plt.subplots(figsize=(6.5, 5.5))
y_pos = np.arange(len(top_hosts))
bars = ax.barh(y_pos, top_hosts.values, height=0.65, color='#2b5c8f', alpha=0.75, edgecolor='#111111', linewidth=0.8, zorder=3)
ax.invert_yaxis()

for bar in bars:
    w = bar.get_width()
    ax.text(w + 0.35, bar.get_y() + bar.get_height()/2, f"{int(w)}", va='center', ha='left', fontsize=8, fontweight='semibold')

ax.set_yticks(y_pos)
ax.set_yticklabels(top_hosts.index, fontsize=8.5)
ax.set_xlabel('Sample Count in Dataset', fontsize=9.5)
ax.set_title('Top 20 Scintillator Host Matrices by Frequency', fontweight='bold', fontsize=10.5, pad=10)
ax.set_xlim(0, max(top_hosts.values) + 4)
ax.grid(axis='x', linestyle=':', alpha=0.5, zorder=0)
plt.tight_layout()
for ext in ['pdf', 'png']:
    plt.savefig(os.path.join(OUTPUT_DIR, f'Figure_S2_compound_frequency.{ext}'), format=ext)
plt.close()

# --- Main Parity Plots (Vertical Publication Stack) ---
SCALE = 1e3
all_vals = np.concatenate([d['y_actual'] for d in results.values()] + [d['y_pred'] for d in results.values()]) / SCALE
val_min, val_max = 0, np.percentile(all_vals, 99.8) * 1.05
panel_labels = ['(a)', '(b)', '(c)']

fig, axes = plt.subplots(len(results), 1, figsize=(3.5, 3.2 * len(results)), sharex=True)
for i, (model_name, data) in enumerate(results.items()):
    ax = axes[i]
    y_act = data['y_actual'] / SCALE
    y_pr = data['y_pred'] / SCALE
    mae_val = mean_absolute_error(data['y_actual'], data['y_pred'])

    ax.plot([val_min, val_max], [val_min, val_max], color='#d62728', linestyle='--', linewidth=1.1, zorder=2)
    ax.scatter(y_act, y_pr, s=18, alpha=0.7, color='#2b5c8f', edgecolors='white', linewidths=0.3, zorder=3)
```

```
261
262     stats_text = f"$R^2 = {data['r2_score']:.3f}$\nRMSE = {data['rmse']:,.0f}\
    nMAE = {mae_val:,.0f}"
263     ax.text(0.95, 0.05, stats_text, transform=ax.transAxes, va='bottom', ha='
    right', fontsize=8, family='monospace',
264             bbox=dict(boxstyle='square,pad=0.35', facecolor='white', edgecolor=
    '#e2e2e2', alpha=0.9))
265
266     ax.set_xlim(val_min, val_max)
267     ax.set_ylim(val_min, val_max)
268     ax.set_aspect('equal', adjustable='box')
269     ax.grid(True, linestyle=':', alpha=0.5, zorder=1)
270     ax.xaxis.set_major_locator(ticker.MultipleLocator(25))
271     ax.yaxis.set_major_locator(ticker.MultipleLocator(25))
272     ax.ticklabel_format(useOffset=False, style='plain')
273     ax.set_title(f"{panel_labels[i]} {model_name}", fontweight='bold', fontsize
    =9.5, pad=8)
274     ax.set_ylabel(r'Predicted LY ($10^3$ ph/MeV)', fontsize=8.5)
275     if i == len(results) - 1:
276         ax.set_xlabel(r'Measured LY ($10^3$ ph/MeV)', fontsize=8.5)
277
278 plt.tight_layout()
279 for ext in ['pdf', 'png']:
280     plt.savefig(os.path.join(OUTPUT_DIR, f'parity_vertical_optimized.{ext}'),
    format=ext, bbox_inches='tight')
281 plt.close()
282
283 # --- Physics Benchmark Figure ---
284 band_gaps = best_data['band_gap']
285 y_actual = best_data['y_actual']
286 y_pred = best_data['y_pred']
287 ly_max_nominal = 1e6 / (2.5 * band_gaps)
288 efficiency_exp = y_actual / ly_max_nominal
289 efficiency_ml = y_pred / ly_max_nominal
290
291 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.5, 3.5))
292 bg_grid = np.linspace(max(1.0, band_gaps.min() * 0.95), 8.5, 500)
293 ly_grid_upper = (1e6 / (2.0 * bg_grid)) / SCALE
294 ly_grid_lower = (1e6 / (3.0 * bg_grid)) / SCALE
295
296 ax1.fill_between(bg_grid, ly_grid_lower, ly_grid_upper, color='#e5e5e5', alpha
    =0.7, zorder=1)
297 ax1.plot(bg_grid, ly_grid_upper, color='#222222', linestyle='-', lw=1.3, zorder
    =2)
298 ax1.plot(bg_grid, ly_grid_lower, color='#555555', linestyle='--', lw=1.3,
    zorder=2)
299 ax1.scatter(band_gaps, y_actual / SCALE, s=20, marker='o', facecolor='#2b5c8f',
     edgecolor='#111111', linewidth=0.5, alpha=0.65, label='Experimental',
    zorder=3)
300 ax1.scatter(band_gaps, y_pred / SCALE, s=26, marker='d', facecolor='none',
    edgecolor='#b33a00', linewidth=1.1, alpha=0.85, label=f'ML ({best_model_name
    })', zorder=4)
301 ax1.set_xlim(0.8, 8.5)
302 ax1.set_ylim(0, 130)
303 ax1.set_xlabel(r'Host Band Gap, $E_g$ (eV)', fontsize=10)
304 ax1.set_ylabel(r'Light Yield ($10^3$ ph/MeV)', fontsize=10)
305 ax1.set_title('(a) Benchmark vs. Fundamental Limit', fontweight='bold',
    fontsize=10.5, pad=8)
306 ax1.grid(True, linestyle=':', alpha=0.45)
307 ax1.legend(frameon=True, fontsize=8, loc='lower left', framealpha=0.92,
    edgecolor='#cccccc')
308
```

```
bins = np.linspace(0, max(np.percentile(efficiency_exp, 99.5), np.percentile(efficiency_ml, 99.5)) * 1.15, 25)
ax2.hist(efficiency_exp, bins=bins, density=True, histtype='stepfilled', facecolor='#2b5c8f', alpha=0.25, hatch='///', edgecolor='#2b5c8f', lw=1.2, label=r'Exp. $\eta_{\mathrm{exp}}$')
ax2.hist(efficiency_ml, bins=bins, density=True, histtype='stepfilled', facecolor='#b33a00', alpha=0.15, hatch=r'\\\\', edgecolor='#b33a00', lw=1.2, label=r'ML $\eta_{\mathrm{ML}}$')
ax2.set_xlabel(r'Efficiency Ratio, $\eta$ ($\mathrm{LY} / \mathrm{LY}_{\mathrm{max,nominal}}$)', fontsize=10)
ax2.set_ylabel('Probability Density', fontsize=10)
ax2.set_title('(b) Scintillation Efficiency Distribution', fontweight='bold', fontsize=10.5, pad=8)
ax2.grid(True, linestyle=':', alpha=0.45)
ax2.legend(frameon=True, fontsize=8, loc='upper right', framealpha=0.92, edgecolor='#cccccc')

plt.tight_layout()
for ext in ['pdf', 'png']:
    plt.savefig(os.path.join(OUTPUT_DIR, f'physics_benchmark_publication.{ext}'), format=ext)
plt.close()

# --- SHAP Summary Plots (Figures S7a & S7b) ---
X_sel_shap = best_data['X_selected'].copy()
feature_name_map = {
    'host_band_gap': r'Host Band Gap, $E_g$',
    'dop_conc': 'Dopant Concentration',
    'density_gcc': r'Density ($\mathrm{g/cm^3}$)',
    'av_ion_pot': 'Mean Ionization Potential',
    'ion_pot+': r'Cation Ionization Pot. ($I^+$)',
    'ion_pot-': r'Anion Ionization Pot. ($I^-$)',
    'av_electroneg': r'Mean Electronegativity ($\bar{\chi}$)',
    'av.ele+': r'Cation Electronegativity ($\chi^+$)',
    'av.ele-': r'Anion Electronegativity ($\chi^-$)',
    'av_elec_aff': 'Mean Electron Affinity',
    'elec_aff+': r'Cation Electron Affinity ($EA^+$)',
    'elec_aff-': r'Anion Electron Affinity ($EA^-$)',
    'emp_rad+': r'Cation Empirical Radius ($r^+$)',
    'emp_rad-': r'Anion Empirical Radius ($r^-$)',
    'electronegativity_mismatch': r'$\Delta\chi$ Mismatch',
    'size_mismatch': r'Ionic Radius Mismatch ($\Delta r$)',
    'bandgap_doping_ratio': r'Ratio $E_g / \mathrm{Conc_{dop}}$'
}
X_sel_shap.rename(columns=lambda c: feature_name_map.get(c, c), inplace=True)

explainer = shap.TreeExplainer(best_data['best_model'])
shap_values = explainer.shap_values(X_sel_shap)

# Figure S7a: Global Importance Bar Plot
fig, ax = plt.subplots(figsize=(4.8, 4.2))
shap.summary_plot(shap_values, X_sel_shap, plot_type="bar", color='#2b5c8f', max_display=10, show=False)
plt.title('Global Feature Importance', fontweight='bold', fontsize=11, pad=10)
plt.xlabel(r'Mean (|SHAP Value|) on $\ln(1 + \mathrm{LY})$', fontsize=9.5)
plt.grid(axis='x', linestyle=':', alpha=0.5, zorder=0)
plt.tight_layout()
for ext in ['pdf', 'png']:
    plt.savefig(os.path.join(OUTPUT_DIR, f'Figure_S7a_feature_importance.{ext}'), format=ext, bbox_inches='tight')
plt.close()

# Figure S7b: Directional Beeswarm Plot
```

```
fig, ax = plt.subplots(figsize=(5.4, 4.2))
shap.summary_plot(shap_values, X_sel_shap, max_display=10, cmap='coolwarm',
    alpha=0.65, show=False)
plt.title('Directional SHAP Value Distribution', fontweight='bold', fontsize
    =11, pad=10)
plt.xlabel(r'SHAP Value (Impact on $\ln(1 + \mathrm{LY})$)', fontsize=9.5)
plt.grid(True, linestyle=':', alpha=0.35, zorder=0)
plt.tight_layout()
for ext in ['pdf', 'png']:
    plt.savefig(os.path.join(OUTPUT_DIR, f'Figure_S7b_shap_beeswarm.{ext}'),
    format=ext, bbox_inches='tight')
plt.close()

print(f"\nPipeline successfully completed! All models trained and publication
    figures rendered to: {OUTPUT_DIR}")
```

References

[1] A. Datta, S. Sen, and Y. Zick, "Algorithmic transparency via quantitative input influence: Theory and experiments with learning systems," in *2016 IEEE Symposium on Security and Privacy (SP)*, pp. 598–617, 2016. https://doi.org/10.1109/SP.2016.42

[2] S. Lipovetsky and M. Conklin, "Analysis of regression in game theory approach," *Applied Stochastic Models in Business and Industry*, vol. 17, no. 4, pp. 319–330, 2001. https://doi.org/10.1002/asmb.446

[3] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 4765–4774, 2017.

[4] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016. https://doi.org/10.1145/2939672.2939778

[5] L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games*, vol. II (AM-28), pp. 307–318, Princeton University Press, 1953.